

TECHNICAL SPECIFICATION

PR Sentry

Architecture & Design Specification — Hackathon Edition

1. Executive Summary

PR Sentry is a Docker-based GitHub Action designed to automatically review C# pull request code additions. Rather than reviewing entire files, it isolates analysis strictly to modified code patches. This prevents noisy warnings on legacy code and guides developers directly to grounded, actionable findings mapped to real new-file line numbers.

2. Architectural Overview

The action operates entirely inside an isolated containerized environment, calling GitHub REST endpoints to read diff data and submit review comments.

- **Node.js 20 & TypeScript:** The core processing engine written under strict type check rules.
- **Docker Environment:** Encapsulates runtime dependencies, ensuring zero external configuration is needed.
- **Octokit Client:** Powers GitHub API calls for fetching PR diff patches, comments list, and posting reviews.

3. Pipeline Workflow

PR Sentry performs a sequential pipeline during a pull request event:

1. **Event Trigger:** Triggered automatically on `pull_request` (types: `opened` , `synchronize`).

2. **Fetch Diff:** Fetches the file modifications via the GitHub PR Files API (`GET /repos/{owner}/{repo}/pulls/{pull_number}/files`).
3. **Filter Files:** Skips non-C# files, deleted C# files, generated C# files (e.g., `.g.cs` , `.designer.cs`), and binary/minified files safely.
4. **Line Mapping:** Parses the unified diff patch to map the added lines (prefixed with `+`) to their exact line numbers in the new file.
5. **Core Analyzers:** Runs the null-safety, async/await correctness, and SOLID SRP scanners.
6. **Deduplication:** Fetches existing PR review comments and filters out duplicate findings to prevent spamming the PR conversation.
7. **Auto-Resolution:** If a previously flagged finding is resolved in a new commit, it automatically updates the old comment body, crossing it out and tagging it as resolved.
8. **Publish Review:** Posts all new findings as a single review event (`event: COMMENT`) with inline comments and a summary table in the review body.

4. Core Static Analyzers

A. Null-Handling Analyzer

Detects common null dereferencing vulnerabilities using high-confidence syntactic patterns:

- **Nullable Return Dereference:** Detects assignments from `FirstOrDefault` , `SingleOrDefault` , `Find` , or `as Type` . It flags any direct dereference (`varName.Member`) in subsequent lines unless a null check (`!= null` , `is not null` , `?.`) is present in the hunk.
- **Explicit Null Dereference:** Detects variables explicitly assigned to `null` that are dereferenced, flagged as severe error.
- **Null Forgiving Operator:** Detects unsafe null-suppression operator usage (`varName!.Member`).

B. Async Correctness Analyzer

Ensures correct async behavior by flagging unsafe asynchronous C# patterns:

- **Async Void:** Flags declarations of `async void` methods, warning that they prevent callers from observing exceptions or waiting.
- **Blocking Calls:** Identifies synchronous blocks on tasks via `.Result` or `.Wait()` on `async` expressions.
- **Synchronous Async Methods:** Flags methods marked with `async` that lack any `await` calls in their body hunk.

C. SOLID SRP Analyzer

Performs Single Responsibility Principle (SRP) heuristic checking:

- Groups added methods under their respective class declarations.
- Maps method names to 5 responsibilities: User Management, Notifications/Mailing, Database/DB, Billing/Invoicing, and Reporting/Exporting.
- If a class combines methods matching **3 or more distinct responsibilities**, it is flagged as a God Class.

5. Threat and Hostile Input Mitigation

To operate securely in public contexts, the runtime implements these safeguards:

- **Size Limits:** Patches exceeding 512KB are skipped to prevent memory depletion.
- **Minified Detection:** Added lines exceeding 5000 characters are flagged as minified and skipped.
- **Binary Detection:** Patches containing null characters or binary markers are skipped.
- **Error Boundaries:** Failures in single file parsing are caught locally, ensuring a corrupted file never crashes the global workflow.